Day 1 Live-Coding Session Solutions

Comprehensive Java implementations for all five coding arcade and verification modules.

1. Rock-Paper-Scissors Game

Simulator playing rounds between player and computer with a final scoreboard summary.

```
      public static String playRound(String playerMove, String computerMove) { if
(playerMove.equalsIgnoreCase(computerMove)) return "Draw"; if
((playerMove.equalsIgnoreCase("Rock") && computerMove.equalsIgnoreCase("Scissors")) ||
(playerMove.equalsIgnoreCase("Paper") && computerMove.equalsIgnoreCase("Rock")) ||
(playerMove.equalsIgnoreCase("Scissors") && computerMove.equalsIgnoreCase("Paper"))) {
return "Player Wins"; } else { return "Computer Wins"; } }
```

Concepts Covered: Random number generation, conditional logic, loops, tabular tracking, and percentage calculations.

2. Palindrome Checker (3 Approaches)

Verifies text consistency using iterative, recursive, and array-reversal implementations.

```
      public static boolean isPalindromeIterative(String text) { int left = 0, right =
text.length() - 1; while (left < right) { if (text.charAt(left) != text.charAt(right))
return false; left++; right--; } return true; } public static boolean
isPalindromeRecursive(String text) { if (text == null || text.length() <= 1) return
true; if (text.charAt(0) != text.charAt(text.length() - 1)) return false; return
isPalindromeRecursive(text.substring(1, text.length() - 1)); }
```

```
      public static boolean isPalindromeArrayReversal(String text) { char[] arr =
text.toCharArray(); String rev = ""; for (int i = arr.length - 1; i >= 0; i--) rev +=
arr[i]; return text.equals(rev); }
```

Concepts Covered: Loops, recursion, array manipulation, string comparison, and implementation trade-offs.

3. BMI Calculator for a Team

Computes BMI and health classifications for an enterprise wellness report.

```
      public static String getBmiStatus(double bmi) { if (bmi < 18.5) return
"Underweight"; else if (bmi <= 24.9) return "Normal"; else if (bmi <= 29.9) return
"Overweight"; else return "Obese"; } public static void printWellnessReport(double[]
heights, double[] weights) { for (int i = 0; i < heights.length; i++) { double bmi =
weights[i] / (heights[i] * heights[i]); System.out.printf("Person %d - BMI: %.2f |
Status: %s%n", (i + 1), bmi, getBmiStatus(bmi)); } }
```

Concepts Covered: Parallel arrays, arithmetic operations, conditional logic, and formatted tabular output.

4. First Non-Repeating Character

Unique letter hunt utility scanning strings left to right using frequency maps.

```
      public static char findFirstNonRepeatingChar(String text) { Map<Character,
Integer> map = new HashMap<>(); for (int i = 0; i < text.length(); i++) { char c =
text.charAt(i); map.put(c, map.getOrDefault(c, 0) + 1); } for (int i = 0; i <
text.length(); i++) { char c = text.charAt(i); if (map.get(c) == 1) return c; } return
'\0'; }
```

Concepts Covered: Character frequency counting, HashMap structures, and early-exit scanning.

5. Reverse Customer Name & Main Execution

Identity verification module reversing strings securely without modifying originals.

```
public static String reverseCustomerName(String name) { char[] arr = name.toCharArray(); String rev = ""; for (int i = arr.length - 1; i >= 0; i--) rev += arr[i]; return rev; }
```

```
public static void main(String[] args) { runRockPaperScissorsDemo(); runPalindromeDemo(); printWellnessReport(new double[]{1.75, 1.60}, new double[]{70.0, 90.0}); runNonRepeatingCharDemo(); System.out.println("Reversed Name: " + reverseCustomerName("Sunil")); }
```

Concepts Covered: String traversal, character array manipulation, and modular program structure.